

INFO-H-415 – Advanced databases

First session examination

A hotel chain has put together a database to keep track of various aspects of their business.

The database is based on the following relational schema:

- **City (CityID, Name, CityGeo, HistoricCenterGeo)**

- CityGeo is a geometrical POLYGON
- HistoricCenterGeo is a geometrical POLYGON

- **Hotel (HotelID, HotelName, HotelGeo, FromDate, ToDate, CityID)**

- HotelGeo is a geometrical POINT corresponding to the address of the hotel
- FromDate is the date of creation
- ToDate is the date of removal (null if still exists)
- CityID references City(CityID)

- **HotelRoom (HotelID, RoomID, MaxPersons)**

- HotelID references Hotel(HotelID)

- **HotelRoomPrice (HotelID, RoomID, PricePerNight, FromDate, ToDate)**

- (HotelID, RoomID) references HotelRoom(HotelID, RoomID)
- FromDate is the date of setting the price
- ToDate is the date of changing the price (maximum date for the current row)

- **RoomReservation (ReservationID, HotelID, RoomID, ReservationDate, NoOfPersons, From-Date, ToDate)**

- (HotelID,RoomID) references HotelRoom((HotelID,RoomID)))
- ReservationDate is the date of the reservation
- FromDate date of check-in
- ToDate date of check-out

- **TouristCircuit (CircuitID, Name, CircuitGeo, CityID)**

- CircuitGeo is a geometrical LINESTRING
- CityID references City(CityID)

- **PointOfInterest (POIID, Name, POIGeo, CityID)**

- POIGeo is a geometrical POINT
- CityID references City(CityID)

1 Spatial Databases

For the following questions, suppose you are using a PostgreSQL database with the POSTGIS extension added. We also suppose working in 4326 (WGS84) spatial reference system.

1. For each city, give the currently most expensive hotel in its historical centre (Justify your computation of "most expensive").

Explanation:

- "Currently" means checking the price where ToDate IS NULL in HotelRoomPrice.
- "Most expensive" is based on the maximum room price.
- The hotel must be inside the historic center (ST_Within).

```
SELECT c.CityID, c.Name AS CityName, h.HotelID, h.HotelName, MAX(p.PricePerNight)  
AS MaxPrice  
FROM City c  
JOIN Hotel h ON h.CityID = c.CityID  
JOIN HotelRoom hr ON hr.HotelID = h.HotelID  
JOIN HotelRoomPrice p ON p.HotelID = hr.HotelID AND p.RoomID = hr.RoomID  
WHERE ST_Within(h.HotelGeo, c.HistoricCenterGeo)  
AND p.ToDate IS NULL  
GROUP BY c.CityID, c.Name, h.HotelID, h.HotelName;
```

```
SELECT c.CityID, c.Name AS CityName, h.HotelID, h.HotelName, MAX(p.PricePerNight)  
AS MaxPrice
```

- SELECT → Specifies the columns to retrieve.
- c.CityID, c.Name AS CityName → Retrieves the city's ID and name (renamed as CityName).
- h.HotelID, h.HotelName → Retrieves the hotel's ID and name.
- MAX(p.PricePerNight) AS MaxPrice → Finds the most expensive room in the hotel by using MAX() on PricePerNight.

FROM City c

- FROM City c → c is an alias for the City table, which contains city-related data.

JOIN Hotel h ON h.CityID = c.CityID

- JOIN Hotel h → Connects Hotel with City using the foreign key CityID.
- Why? → Each hotel belongs to a specific city.

JOIN HotelRoom hr ON hr.HotelID = h.HotelID

- JOIN HotelRoom hr → Links HotelRoom to Hotel using HotelID.
- Why? → This ensures that we can later fetch room prices for each hotel.

JOIN HotelRoomPrice p ON p.HotelID = hr.HotelID AND p.RoomID = hr.RoomID

- JOIN HotelRoomPrice p → Connects room pricing information to hotel rooms.
- ON p.HotelID = hr.HotelID AND p.RoomID = hr.RoomID → Ensures we match the correct room prices to each hotel room.

WHERE ST_Within(h.HotelGeo, c.HistoricCenterGeo)

- ST_Within(h.HotelGeo, c.HistoricCenterGeo) → PostGIS function that checks if the hotel location (POINT) is inside the historic center (POLYGON) of the city.
- Why? → We only want hotels located in the historical center.

AND p.ToDate IS NULL

- AND p.ToDate IS NULL → Ensures that only the currently active price is considered (ignores outdated prices).
- Why? → If ToDate IS NULL, it means the price is still valid.

GROUP BY c.CityID, c.Name, h.HotelID, h.HotelName;

- GROUP BY → Groups results by city and hotel.
- Why? → Since MAX(p.PricePerNight) is used, we group by city and hotel to find the highest price for each hotel.

1 Spatial Databases

2. For each city, rank their tourist circuit by the number of points of interest. A point of interest is part of the tourist circuit if it is less than 100m away from the circuit.

Explanation:

- A POI is considered part of a tourist circuit if it's within 100 meters of the circuit (ST_Distance < 100).
- We count the POIs and rank circuits per city.

```
SELECT c.CityID, tc.CircuitID, tc.Name AS CircuitName, COUNT(p.POIID) AS POICount,  
RANK() OVER (PARTITION BY c.CityID ORDER BY COUNT(p.POIID) DESC) AS RankInCity  
FROM City c  
JOIN TouristCircuit tc ON tc.CityID = c.CityID  
JOIN PointOfInterest p ON p.CityID = c.CityID  
WHERE ST_Distance(p.POIGeo, tc.CircuitGeo) < 100  
GROUP BY c.CityID, tc.CircuitID, tc.Name;
```

```
SELECT c.CityID, tc.CircuitID, tc.Name AS CircuitName, COUNT(p.POIID) AS POICount
```

- c.CityID → Selects the City ID (to group results per city).
- tc.CircuitID → Selects the Tourist Circuit ID (to identify each circuit).
- tc.Name AS CircuitName → Retrieves the circuit name (renamed as CircuitName).
- COUNT(p.POIID) AS POICount → Counts the number of Points of Interest (POIs) near each circuit.

```
RANK() OVER (PARTITION BY c.CityID ORDER BY COUNT(p.POIID) DESC) AS RankInCity
```

- RANK() OVER (...) AS RankInCity → Assigns a rank to each circuit in a city based on how many POIs it has.
- PARTITION BY c.CityID → Resets the ranking for each city separately.
- ORDER BY COUNT(p.POIID) DESC → Ranks circuits in descending order (most POIs = Rank 1).

```
FROM City c
```

```
JOIN TouristCircuit tc ON tc.CityID = c.CityID
```

```
JOIN PointOfInterest p ON p.CityID = c.CityID
```

- FROM City c → The main table is City (c is an alias).
- JOIN TouristCircuit tc ON tc.CityID = c.CityID Joins TouristCircuit (tc) to City (c) using CityID.
- Ensures that we only analyze circuits belonging to that city.
- JOIN PointOfInterest p ON p.CityID = c.CityID Joins PointOfInterest (p) to City (c) using CityID.
- Ensures that we only consider POIs within the same city as the circuits.

WHERE ST_Distance(p.POIGeo, tc.CircuitGeo) < 100

- ST_Distance(p.POIGeo, tc.CircuitGeo) < 100 PostGIS function that calculates the distance between:
 - p.POIGeo (POINT) → Location of the POI.
 - tc.CircuitGeo (LINESTRING) → Path of the tourist circuit.
- Condition → Only include POIs that are within 100 meters of the circuit.

GROUP BY c.CityID, tc.CircuitID, tc.Name;

GROUP BY ensures:

- We count POIs for each circuit.
- We rank circuits within each city.

3. For each tourist circuit, compute the time it will take to complete. Suppose a walking pace of 5km/h and each point of interest on the circuit takes 20min.

- Walking pace = 5 km/h.
- Each POI on the circuit (<100m from the route) adds 20 minutes.
- The length of the circuit (ST_Length) gives the distance in degrees (since SRID 4326 is geographic), so we'd use ST_Transform to convert it into meters or kilometers.
- ST_Transform(tc.CircuitGeo, 3857) changes coordinates to meters (SRID 3857).
- / 1000 converts meters to kilometers.
- / 5 * 60 converts kilometers to minutes at 5 km/h.

SELECT tc.CircuitID, tc.Name,

ROUND((ST_Length(ST_Transform(tc.CircuitGeo, 3857)) / 1000) / 5 * 60, 2) AS

WalkingTimeMinutes,

COUNT(DISTINCT p.POIID) * 20 AS POITimeMinutes,

ROUND((ST_Length(ST_Transform(tc.CircuitGeo, 3857)) / 1000) / 5 * 60 +

COUNT(DISTINCT p.POIID) * 20, 2) AS TotalTimeMinutes

FROM TouristCircuit tc

LEFT JOIN PointOfInterest p

ON tc.CityID = p.CityID

AND ST_Distance(p.POIGeo, tc.CircuitGeo) < 100

GROUP BY tc.CircuitID, tc.Name, tc.CircuitGeo;

1 Spatial Databases - Query 3

SELECT tc.CircuitID, tc.Name,

- tc.CircuitID → Selects the unique ID of each tourist circuit.
- tc.Name → Selects the name of each circuit.

**ROUND((ST_Length(ST_Transform(tc.CircuitGeo, 3857)) / 1000) / 5 * 60, 2) AS
WalkingTimeMinutes,**

This calculates the walking time in minutes, assuming 5 km/h speed. Let's break it down:

- ST_Length(tc.CircuitGeo) → Computes the total length of the circuit (LINESTRING).
- ST_Transform(tc.CircuitGeo, 3857) → Converts the circuit's geometry from SRID 4326 (WGS84 - degrees) to 3857 (meters).
 - Why? SRID 4326 stores geographic coordinates in degrees, so to work with distances in meters, we transform it to SRID 3857.
- / 1000 → Converts meters to kilometers.
- / 5 → Since the walking pace is 5 km/h, this gives time in hours.
- * 60 → Converts hours into minutes.
- ROUND(..., 2) → Rounds the result to 2 decimal places.

Example Calculation:

- If the circuit length is 10,000 meters (10 km): $(10 \text{ km} / 5 \text{ km/h}) * 60 = (2 \text{ hours}) * 60 = 120 \text{ mins}$

COUNT(DISTINCT p.POIID) * 20 AS POITimeMinutes,

- COUNT(DISTINCT p.POIID) → Counts the unique Points of Interest (POI) near the circuit.
- * 20 → Since each POI visit takes 20 minutes, this gives the total time spent visiting POIs.

Example Calculation:

- If a circuit has 3 POIs nearby, the POI visit time is: $3 * 20 = 60 \text{ minutes}$

**ROUND((ST_Length(ST_Transform(tc.CircuitGeo, 3857)) / 1000) / 5 * 60 +
COUNT(DISTINCT p.POIID) * 20, 2) AS TotalTimeMinutes**

This simply adds up the walking time and POI visit time.

Example Calculation: Walking Time = 120 minutes, POI Visit Time = 60 minutes

Total Time = $120 + 60 = 180 \text{ minutes}$

1 Spatial Databases

```
FROM TouristCircuit tc  
LEFT JOIN PointOfInterest p  
ON tc.CityID = p.CityID  
AND ST_Distance(p.POIGeo, tc.CircuitGeo) < 100
```

- LEFT JOIN PointOfInterest p → Joins each tourist circuit with the POIs in the same city.
- ST_Distance(p.POIGeo, tc.CircuitGeo) < 100 Ensures that only POIs within 100 meters of the circuit are considered.
- This filters out distant POIs that aren't relevant.

```
GROUP BY tc.CircuitID, tc.Name, tc.CircuitGeo;
```

```
--> GROUP BY tc.CircuitID, tc.Name, tc.CircuitGeo
```

- Ensures that we compute walking time and POI counts per circuit.
- Since we use aggregates (COUNT(), ST_Length()), we must group by the circuit.

4. For each hotel, list the tourist circuit that have their starting and ending points less than 1km away from the hotel.

Explanation:

- Get circuits where both:
- StartPoint is within 1 km.
- EndPoint is within 1 km.
- Again, since 4326 is in degrees, we use ST_Transform to work in meters.

```
SELECT h.HotelID, h.HotelName, tc.CircuitID, tc.Name AS CircuitName  
FROM Hotel h  
JOIN TouristCircuit tc ON h.CityID = tc.CityID  
WHERE ST_Distance(  
    ST_Transform(h.HotelGeo, 3857),  
    ST_Transform(ST_StartPoint(tc.CircuitGeo), 3857) ) < 1000  
AND ST_Distance(  
    ST_Transform(h.HotelGeo, 3857),  
    ST_Transform(ST_EndPoint(tc.CircuitGeo), 3857) ) < 1000;
```

1 Spatial Databases - Query 4

SELECT h.HotelID, h.HotelName, tc.CircuitID, tc.Name AS CircuitName

- h.HotelID, h.HotelName → Selects the hotel's ID and name.
- tc.CircuitID, tc.Name AS CircuitName → Selects the tourist circuit's ID and name, renaming it as CircuitName.

FROM Hotel h

JOIN TouristCircuit tc ON h.CityID = tc.CityID

- FROM Hotel h → Starts with the Hotel table.
- JOIN TouristCircuit tc ON h.CityID = tc.CityID
 - Joins the TouristCircuit table (tc) with Hotel (h).
 - Why? → Ensures that we only compare hotels and circuits in the same city.

WHERE ST_Distance(

ST_Transform(h.HotelGeo, 3857),

ST_Transform(ST_StartPoint(tc.CircuitGeo), 3857)) < 1000

- ST_StartPoint(tc.CircuitGeo) → Extracts the starting point of the circuit.
- ST_Transform(ST_StartPoint(tc.CircuitGeo), 3857) → Converts this point from SRID 4326 (degrees) to SRID 3857 (meters).
- ST_Transform(h.HotelGeo, 3857) → Converts the hotel's location from degrees to meters.
- ST_Distance(...) → Measures the distance between the hotel's location and the circuit's start point.
- < 1000 → Ensures the hotel is within 1000 meters (1 km) of the start point

AND ST_Distance(

ST_Transform(h.HotelGeo, 3857),

ST_Transform(ST_EndPoint(tc.CircuitGeo), 3857)) < 1000;

- ST_EndPoint(tc.CircuitGeo) → Extracts the ending point of the circuit.
- ST_Transform(ST_EndPoint(tc.CircuitGeo), 3857) → Converts it to meters (SRID 3857).
- ST_Distance(...) < 1000 → Ensures the hotel is also within 1000 meters (1 km) of the end point.

2 TemporalDatabases

For the following questions, suppose you are using a PostgreSQL database (No extensions apart from PostGIS were added!).

1. For each hotel, give the history of the number of unused rooms.

Approach:

- Get the total number of rooms for each hotel.
- Count the occupied rooms at different time points.
- Subtract occupied rooms from total rooms to get unused rooms.

WITH OccupiedRooms AS (

SELECT

rr.HotelID,

rr.FromDate,

rr.ToDate,

COUNT(*) AS Occupied

FROM RoomReservation rr

GROUP BY rr.HotelID, rr.FromDate, rr.ToDate

),

TotalRooms AS (

SELECT

h.HotelID,

COUNT(hr.RoomID) AS TotalRooms

FROM Hotel h

JOIN HotelRoom hr ON h.HotelID = hr.HotelID

GROUP BY h.HotelID

)

SELECT

tr.HotelID,

orr.FromDate,

orr.ToDate,

(tr.TotalRooms - COALESCE(orr.Occupied, 0)) AS UnusedRooms

FROM TotalRooms tr

LEFT JOIN OccupiedRooms orr ON tr.HotelID = orr.HotelID

ORDER BY tr.HotelID, orr.FromDate;